Lesson 12

Arrays and Pointers

Readings: Textbook section 3.0 Example 5 and 6

Slides – Vybihal & Bérubé-Vallières © 2023

Memory

- Memory is divided into bytes (8 bits)
- Each byte has an address

Address	Contents
0	01001010
1	01001101
2	10100010
…	
n	11010100

- Each variable in a program occupies one or more bytes in memory
  - Example: 4-bytes int i located at address 200

Address	Contents
200	
201	
202	
203	
…	

i

- The first byte is the address of the variable
- You will see memory in more detail in COMP273

Slides – Vybihal & Bérubé-Vallières © 2023

Arrays

Arrays

- Up until now, we have seen variables that contain a single data item
- Arrays are a way to aggregate many items that have the same type
  - Next week, we will see structures, which are a way to aggregate many items that have different types
- The individual elements of an array can be selected based on their position in the array

One-Dimensional Arrays

- The simplest and most common array has one dimension.
- Visualized as elements one after another in a single row.
- To declare an array, we follow the type declaration by [SIZE], where SIZE is the size of the array:
- Syntax:
  - TYPE NAME [SIZE];
    - int a[10];
    - char name[30];

Slides – Vybihal & Bérubé-Vallières © 2023

Subscripting (Indexing)

- To access an element, we follow the array name by an integer in square brackets
- Each element is treated the same way as a variable of the type of the array
- Indexing starts at 0

```
int a[10];
```

	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
a										

```
a[0] = 9;
```

	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
a	9									

```
printf("%d", a[0]); // this will output 9
```


Slides – Vybihal & Bérubé-Vallières © 2023

Initializing

- When declaring an array inside a function, the default values are undefined (more later…)
- We have seen how to declare a specific element
  - c[5] = ‘c’;
- We can declare the full array when initializing using an array initializer:
  - int a[5] = {4, 3, 5, 1, 2}

	a[0]	a[1]	a[2]	a[3]	a[4]
a	4	3	5	1	2

- When an initializer is shorter than the array, the rest is given 0
  - int a[5] = {4, 3}

	a[0]	a[1]	a[2]	a[3]	a[4]
a	4	3	0	0	0

- As such, this is how an array is initialized to 0
  - int a[5] = {0}

	a[0]	a[1]	a[2]	a[3]	a[4]
a	0	0	0	0	0

- When array initializer is used, size can be omitted
  - int a[] = {4, 3, 5, 1, 2}

	a[0]	a[1]	a[2]	a[3]	a[4]
a	4	3	5	1	2

Slides – Vybihal & Bérubé-Vallières © 2023

Examples

- Printing the elements of an array

```
int a[5] = {5, 4, 3, 2, 1};

for (int i=0; i<5; i++)
    printf("%d ", a[i]);
```

- Find the index of an element

```
int a[5] = {5, 4, 3, 2, 1};
int search = 3;
int index = -1;

for (int i=0; i<5; i++)
    if (a[i] == search)
        index = i;

if (index != -1)
    printf("The index of %d is %d\n", search, index);
else
    printf("%d is not present in the array\n", search);
```

Accessing uninitialized values

The compiler will not prevent you from accessing a value that has not been initialized: This is **undefined behaviour**

```
int a[5];

printf("%d", a[0]); // what gets printed?? Undefined behaviour
```

Let's say that the following elements are present at memory address 100 (using 4 bytes int) and they are not reserved

100	104	108	112	116
9	4	6	9	5

If you declare int a[5], then the compiler might decide to store it at address 100 and reserve up to 116 for it. But it doesn't have to replace the values, you're not supposed to access them! If it doesn't, then:

```
int a[5];

printf("%d", a[0]); // In the situation above, it might unexpectedly
   print 9!!
```

Be careful to avoid errors like these: Your code might seem like it works sometimes but it will not work other times!

Accessing out-of-bounds values

- Similarly, it will not prevent you from accessing values out of bounds: This is also undefined behaviour.
- Sometimes you might get a segmentation fault (error when you access a memory location you do not have permission for), sometimes you might get unexpected output, etc. An example of a confusing mistake that could happen is this:

```
int a[5];
int i;
for (i = 1; i<=5; i++)
   a[i] = 0;
```

- If i is next to a[4] in memory, then a[5] = 0; will set i to 0, which will restart the loop and cause an infinite loop.

sizeof

- Remember sizeof from previous class

```
int i;
size_t size = sizeof(i);
printf("The size of variable i is %zu bytes\n", size);
   //On mimi, outputs "The size of variable i is 4 bytes"
```

- You can also use it for arrays. The size of an array is the size of the base type times the length
  - Because of this, you can get the length with sizeof(array)/sizeof(array[0])

```
int i[5];
printf("The array itself has size %zu\n", sizeof(i));
    // On mimi, outputs "The array itself has size 20"
 printf("Its elements have size %zu\n", sizeof(i[0]));
   // On mimi, outputs "Its elements have size 4"
printf("The length of the array is %zu\n", sizeof(i)/sizeof(i[0]));
   // Outputs "The length of the array is 5"
```

sizeof returns an unsigned integer of some length, but which specific one depends on compiler/machine (For example on mimi it is a long unsigned integer). C provides a type alias size_t for whichever specific unsigned integer data type is used by sizeof. Its format specifier is %zu

Slides – Vybihal & Bérubé-Vallières © 2023

Multi-Dimensional Arrays

- An array may have multiple dimensions

Syntax:

- TYPE NAME [ROWS][COLS];
  - int picture[100][200];
- TYPE NAME [ROWS][COLS][LAYERS];
  - char world[100][100][50];
- etc.

Initializers can be used with multi-dimensional arrays!
int a[3][5] = {{5, 4, 3, 1, 3},
{5, 2, 5, 6, 4},
{7, 4, 6, 4, 2}};

Example: int a[3][5]

To access element at row i column j, we write a[i][j]

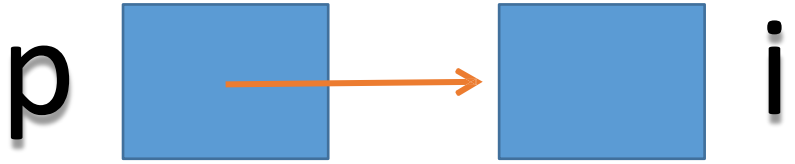
	0	1	2	3	4
0					
1					
2					

In reality, in memory the elements are stored continuously in row order, i.e. a[0,0], …, a[0,4], a[1,0], …, a[1, 4], a[2,0], …, a[2,4]

Pointers

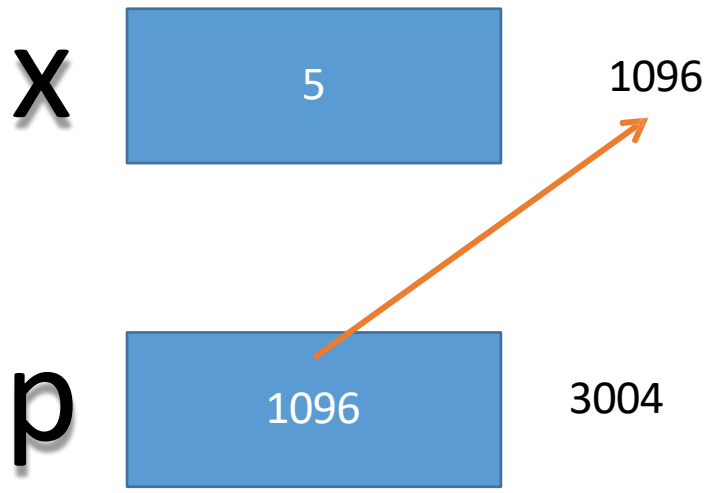
Pointer

- A pointer variable stores the address of a variable.
  - Instead of storing the address, based on compiler could also be an offset to get the address starting from some base address
- If we have a variable i, and we store its address in pointer variable p, we say that p points to i



Example

Let's say that we have an int variable that contains 5 and its address is 1096. Then pointer p contains the location of x. Note how the pointer itself has its own address



Declaring a pointer variable

To declare a pointer variable, the identifier is preceded by an asterisk in the declaration.

Syntax:

```
TYPE *VARIABLE;
```

Example:

```
int *p;
```

This means that p is a pointer variable that can point to an object of type int

Address-of operator (&)

- The address-of operator (&) returns the address of a variable.
- As such, to make a pointer point to a variable, we do the following:

```
int i;
int *p;
p = &i;
// We could also do it right away:
int i;
int *p = &i;
```

- Very useful to pass a variable itself to a function (we will see this in detail next week, but we've quickly seen it before with scanf)

Dereference operator (*)

Also called indirection operator. When a pointer variable points to an object, we can use * to access what is stored in the object. For example

```
int i;
int *p;
p = &i;
i = 5;
printf("%d", i); // prints 5 to the screen
printf("%d", *p); // prints 5 to the screen
*p = 2;
printf("%d", i); // prints 2 to the screen
printf("%d", *p); // prints 2 to the screen
```

Possible confusion

- A common source of confusion is that the same symbol (*) is used for two different things: when declaring a pointer type and when dereferencing.
  - int *p; // Create a pointer “p” that points to an int
  - …
  - p = &i // setting p to the address of some variable i
  - *p = 5 // dereference p to get the object it points to (i) and set it to 5
- One way to make sense of this is to view the variable declaration as this:

int *p means that *p is an int, i.e. when you dereference p you get an int

Good to know

- Note that if you look at examples online (or elsewhere in this course sometimes), the following notation might be used:
  - int* p; // (instead of int *p)
- This is completely equivalent (and so is int * p), whitespace doesn't matter in C
- int* p emphasizes "the type of p is a pointer-to-int" while int *p emphasizes "the dereferenced value of p is an int" (like last slide).
  - int* p is often called the C++ style and int *p the C style
- Both are valid, but you need to be careful to not run in the following issue with int*:
  - int* p, q // Here you might expect p and q to be two pointers, but q is an int! You also need * somewhere before q to get two pointers
  - Compare this to the second way where int *p, *q is more natural
  - People using the first style (int*) usually simply never declare multiple pointers at the same time

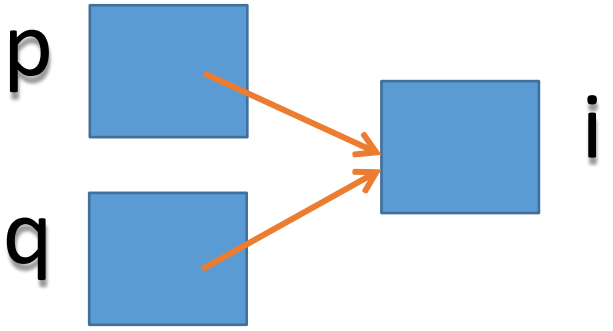
More on pointer assignment

We have seen that we can initialize a pointer like so:

```
int i
int *p;
p = &i; // p points to i
```

You can also set it to the value of another pointer (i.e. the address it contains)

```
int *q;
q = p; // q points to i
*p = 5; // i is now equal to 5
*q = 3; // i is now equal to 3
```



More on printing

As a recap…

```
int i = 5;
int *p = &i;
printf("%d", *p); // prints 5 to the screen
```

What does the following do?

```
printf("%d", p); // might happen by mistake
```

This will automatically try to cast the pointer to an int, but the address might be a larger number than the int size, in which case it will give the decimal representation of the last bytes of the address, which is not useful

Exercise

Write a C program named login.c that performs the following steps:

- The program receives a username as a command-line argument.
- It searches for this username in a predefined list of usernames.
- If the username exists, the program prompts the user to enter a password.
- If the entered password matches the stored password for that username, the program prints: Login successful. Otherwise, it prints: Login valid

Hint: Starter code exist in https://gitlab.cs.mcgill.ca/jerrin/comp-206-notes/-/tree/main/c-programs/oct-10

A solution is also provided (login-complete.c), but it's recommended to try it on your own first.